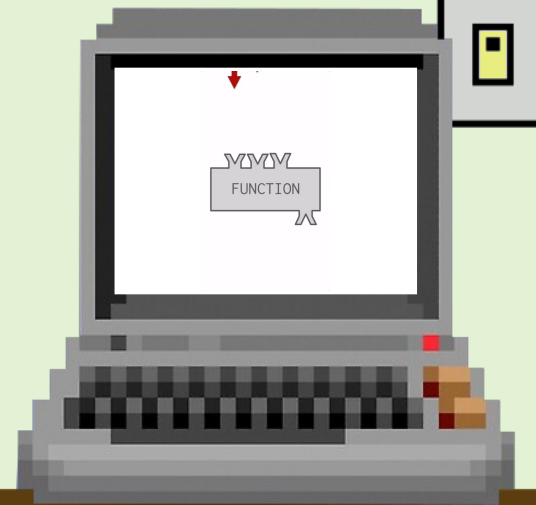


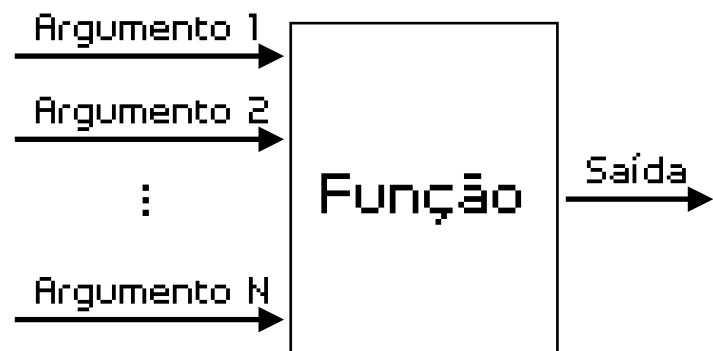
Funções em Haskell

Programação Funcional



Funções

- Definição semelhante à definição matemática
- Associa valores de entrada a uma saída



Exemplos

- Função que calcula as raízes de uma equação
- Função que emite relatório com notas e status dos alunos
- Função que controla a velocidade de um veículo



Programando Funções

- Funções podem ser declaradas sem explicitar seu tipo
 - `<nome_função> <list_args> '=' <expressão>`
 - Ex: `double x = x + x`
- O próprio compilador irá inferir o tipo da função:
 - O tipo da função será igual ao da expressão
 - O argumento será válido para todos os tipos sobre o qual o operador em questão (+) seja válido
 - É aconselhado declarar a função explicitamente



Programando Funções

- Pode ter os tipos de seus argumentos e seu retorno declarado explicitamente através de cabeçalho
 - Exemplo

```
double::Int->Int  
double x = x + x
```



Programando Funções

- Ex: `todosIguais::Int->Int->Int->Bool`
 - Função com três argumentos de entrada do tipo inteiro (Int)
 - O resultado da função é um valor booleano (Bool)

```
todosIguais::Int->Int->Int->Bool  
todosIguais p s t = (p == s) && (s == t)
```



Programando Funções

- Assim como na matemática é comum definir funções a partir de outras (Composição de funções)
- Exemplo
 - Função `somaDobro` em termos da função `dobro`

```
dobro: Int -> Int  
dobro x = x + x
```

```
somaDobro: Int -> Int -> Int  
somaDobro x y = dobro x + dobro y
```



Programando Funções

- Funções "currificadas"
 - Uma função com n argumentos equivale a n funções com um argumento
 - Exemplo

```
todosIguaisPI = todosIguais 3.14
```

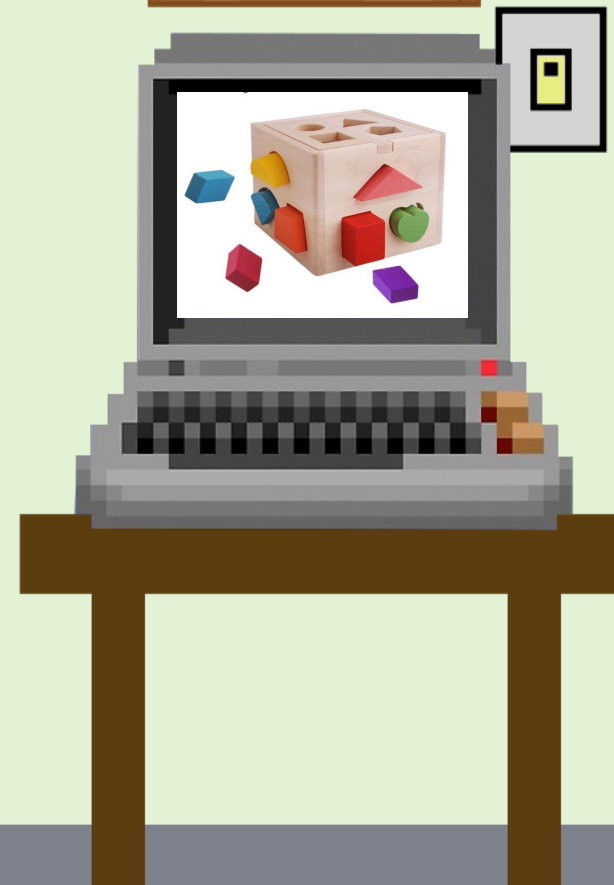
```
todosIguaisPI::Int->Int->Bool  
todosIguaisPI s t = todosIguais 3.14 s t
```



Pattern Matching (Casamento de Padrões)

- Forma de codificar funções baseadas na definição de funções por enumeração de casos
 - São enumerados os valores que os argumentos podem assumir
 - Resultado correspondente declarado em cada caso
 - São exibidos todos os pares (entrada, resultado)
 - Padrões testados em sequência (cima para baixo)
- Exemplo

```
ehZero::Int->Bool  
ehZero 0 = True  
ehZero _ = False
```



Pattern Matching (Casamento de Padrões)

■ Exercício - Definir Fatorial

$$n! = \begin{cases} 1 & \text{if } n=0 \\ n \cdot (n-1)! & \text{if } n>0 \end{cases}$$

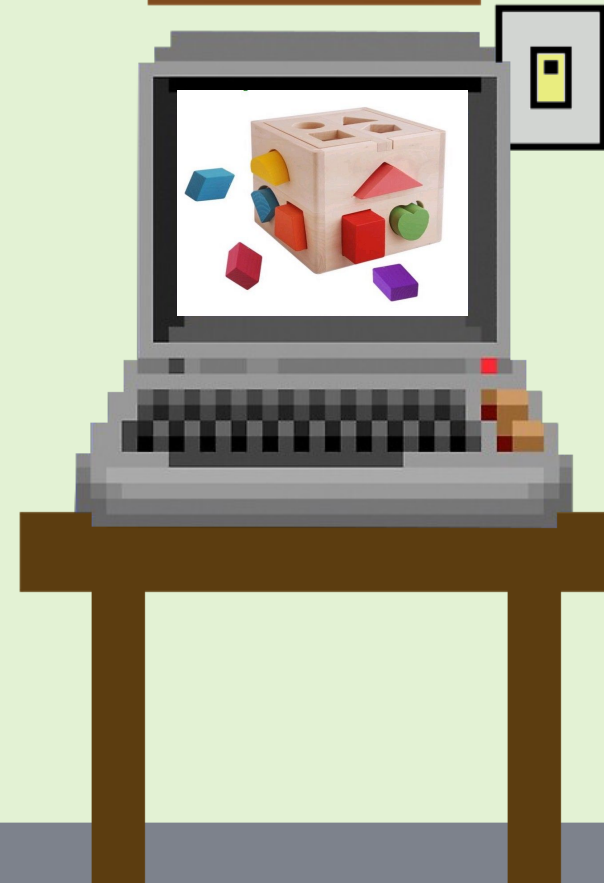


Pattern Matching (Casamento de Padrões)

■ Exercício - Definir Fatorial

$$n! = \begin{cases} 1 & \text{if } n=0 \\ n \cdot (n-1)! & \text{if } n>0 \end{cases}$$

```
Fatorial::Int->Int  
Fatorial 0 = 1  
Fatorial n = n * Fatorial(n-1)
```



Avaliação Condicional com "Guardas"

- Casamento de padrões nos limita a executar testes fixos do formato de um valor
- Haskell possui o mecanismo de "guardas" (checagem antes da execução do corpo de uma função)
- Um padrão pode ser seguido por zero ou mais guardas
 - Cada guarda com uma expressão do tipo booleano
 - Definida com o pipe " | " e a expressão de guarda
 - O símbolo " = " separa o código executado caso a expressão seja True
 - Casos não previstos caem na cláusula "otherwise"



Avaliação Condicional com "Guardas"

■ Exemplo

```
ehZero::Int->Bool
ehZero 0 = True
ehZero _ = False
```

```
ehZero::Int->Bool
ehZero x
  | x==0 = True
  | otherwise = False
```



Avaliação Condicional com "Guardas"

- Exercício - Definir Fatorial

$$n! = \begin{cases} 1 & \text{if } n=0 \\ n \cdot (n-1)! & \text{if } n>0 \end{cases}$$



Avaliação Condicional com "Guardas"

■ Exercício - Definir Fatorial

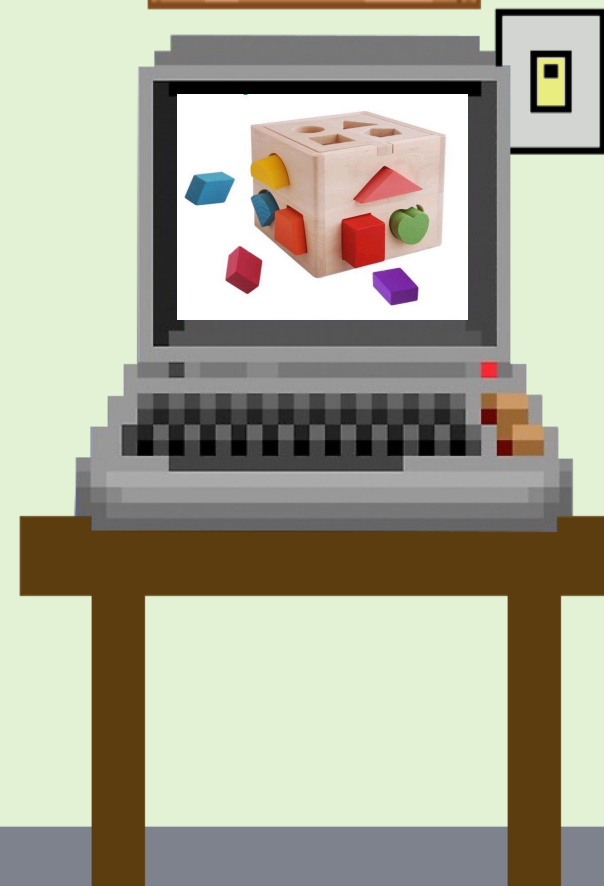
$$n! = \begin{cases} 1 & \text{if } n=0 \\ n \cdot (n-1)! & \text{if } n>0 \end{cases}$$

```
Fatorial::Int->Int
```

```
Fatorial n
```

```
  | n==0 = 1
```

```
  | n > 0 = n * Fatorial (n-1)
```



Definições Locais

- Podemos realizar definições de identificadores locais com a palavra-reservada "where"
- Exemplo

```
somaQuadrados::Int->Int->Int  
somaQuadrados n m = quadN + quadM  
  where  
    quadN = n * n  
    quadM = m * m
```



Avaliações de Funções

- Haskell utiliza a ordem normal de avaliação
 - Leftmost-outermost
- Associado a um mecanismo preguiçoso
 - Avalia apenas quando necessário e apenas uma vez
- Semelhante a avaliação curto-circuito tradicional
- Exemplo

```
todosIguais (quadrado 3) valor (quadrado 2)
= ((quadrado 3) == valor) && (valor == (quadrado 2))
= ((3 * 3) == valor) && (valor == (quadrado 2))
= (9 == valor) && (valor == (quadrado 2))
= (9 == 39) && (39 == (quadrado 2))
= False && (39 == (quadrado 2))
= False
```



Avaliações de Funções

- Exemplo - Função mmc a partir de mdc e div

```
mdc::Int->Int->Int
```

```
mdc x y
```

```
| x == y = x
```

```
| x > y = mdc x-y y
```

```
| otherwise = mdc y x
```

```
div::Int->Int->Int
```

```
div x y
```

```
| x == y = 1
```

```
| x > y = 1 + div x-y y
```

```
| otherwise = 0
```

```
mmc::Int->Int->Int
```

```
mmc x y = div x*y (mdc x y)
```



Exercício

- Defina a função Fibonacci e escreva a sequência de cálculos para Fibonacci(9).

$$f(n) = \begin{cases} n = 0 & 0 \\ n = 1 & 1 \\ n > 1 & f(n-1) + f(n-2) \end{cases}$$

